An application requires users to input their email addresses for registration. Write a program that checks if the given email address is valid. A valid email should contain an "@" symbol and a "." after the "@" with at least one character in between. The program should print "Valid email" or "Invalid email" based on the input.

Sample input :

String email = "user@example.com";

Sample Output :

"Valid email“

```
public class EmailValidator {
   public static void main(String[] args) {
      String email = "user@example.com"; // Sample input
      if (isValidEmail(email)) {
         System.out.println("Valid email");
      } else {
         System.out.println("Invalid email");
      }}
   public static boolean isValidEmail(String email) {
      int atPosition = email.indexOf("@");
      int dotPosition = email.lastIndexOf(".");
      if (atPosition < 1) return false; // there should be at least one character before @
      if (dotPosition < atPosition + 2) return false; // there should be at least one
character between @ and .
      if (dotPosition + 2 >= email.length()) return false; // there should be at least one
character after the .
      return true;
   } }
```

A system needs to format entered phone numbers to a standard format before storing them in a database. The standard format is "(AreaCode) PhoneNumber", where "AreaCode" is the first 3 digits, and "PhoneNumber" is the remaining 7 digits. Write a program that takes a 10-digit phone number as input and formats it accordingly.

Sample input:

String phoneNumber = "1234567890";

Sample output:

"(123) 456-7890"

```
import java.util.Scanner;
public class PhoneNumberFormatter {
   public static void main(String[] args)
 {
      Scanner scanner = new Scanner(System.in);
      System.out.println("Enter a 10-digit phone number:");
      String phoneNumber = scanner.nextLine();
      if (phoneNumber.length() == 10 && phoneNumber.matches("[0-9]+")) {
         String areaCode = phoneNumber.substring(0, 3);
         String firstPart = phoneNumber.substring(3, 6);
         String secondPart = phoneNumber.substring(6);
         String formattedNumber = "(" + areaCode + ") " + firstPart + "-" + secondPart;

         System.out.println(formattedNumber);
      } else {
         System.out.println("Invalid phone number. Please enter a 10-digit phone
number.");
      } }}
```

Develop a program for a website's registration page that checks the strength of the entered password. A strong password must have at least 8 characters, contain at least one uppercase letter, one lowercase letter, one number, and one special character (e.g., @, #, $, etc.). The program should evaluate the password and print "Strong password" if it meets all criteria or "Weak password" otherwise.

Sample input: String password = "Example@123";

Sample output: "Strong password"

```
public class PasswordStrengthChecker {
  public static void main(String[] args) {
    String password = "Example@123";
    System.out.println(checkPasswordStrength(password));
  }
  public static String checkPasswordStrength(String password) {
    if (password.length() < 8) {
      return "Weak password";
    }
    boolean hasUppercase = false;
    boolean hasLowercase = false;
    boolean hasDigit = false;
    boolean hasSpecialChar = false;
    for (int i = 0; i < password.length(); i++) {
      char ch = password.charAt(i);
      if (Character.isUpperCase(ch)) hasUppercase = true;
      if (Character.isLowerCase(ch)) hasLowercase = true;
      if (Character.isDigit(ch)) hasDigit = true;
      if (!Character.isLetterOrDigit(ch)) hasSpecialChar = true;
      // If all conditions are met, no need to continue checking
      if (hasUppercase && hasLowercase && hasDigit && hasSpecialChar) {
        return "Strong password";
      }
    }
    return "Weak password";
  } }
```

In a school system, you have Person as a base class with common attributes like name and age. Derive two classes from it: Student and Teacher. Student has properties like gradeLevel and favoriteSubject, while Teacher has properties like subjectTaught and yearsOfExperience. Create methods in these subclasses to display the person's details along with their specific information.

Sample Input :

Role: Student
Name: "Alice Johnson"
Age: 12
Grade Level: 7
Favorite Subject: "Mathematics“

Expected Output :

Name: Alice Johnson
Age: 12
Grade Level: 7
Favorite Subject: Mathematics

```
class Person {
   String name;
   int age;
   Person(String name, int age) {
      this.name = name;
      this.age = age;
   }
}
class Student extends Person {
   int gradeLevel;
   String favoriteSubject;
   Student(String name, int age, int gradeLevel, String favoriteSubject) {
      super(name, age);
      this.gradeLevel = gradeLevel;
      this.favoriteSubject = favoriteSubject;
   }
   void display() {
      System.out.println("Name: " + name + "\nAge: " + age + "\nGrade Level: " + gradeLevel + "\nFavorite Subject: " +
favoriteSubject);
   }
}
public class SchoolSystem {
   public static void main(String[] args) {
      Student student = new Student("Alice Johnson", 12, 7, "Mathematics");
      student.display();
   }
}
```

Imagine an online retail store where you have a base class Product with properties such as productId, productName, and price. Create two subclasses: Electronics with additional properties like warrantyPeriod and ElectronicType (e.g., mobile, laptop, TV), and Clothing with properties like size, material, and color. Each subclass should have methods to display the full details of the product including the base class properties.